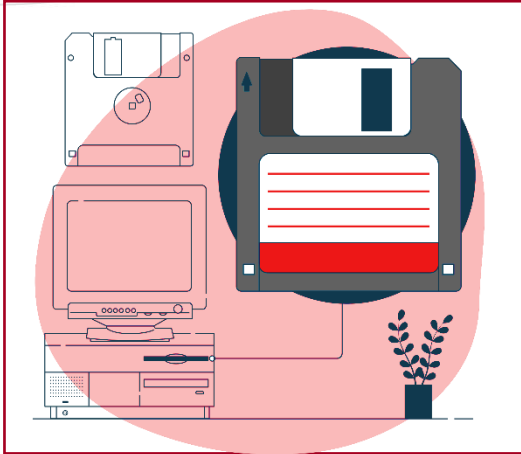




Universidad de El Salvador

Facultad Multidisciplinaria de Occidente
Ingeniería en Desarrollo de Software

Asignatura: **Administración de Bases de Datos**

Unidad #3

Fecha: 08 al 14 de septiembre 2025

Tema: Configuración y administración del espacio en disco.

Introducción:

Este material tiene como objetivo presentarte los conceptos fundamentales sobre el almacenamiento interno de datos en un sistema de gestión de bases de datos (DBMS). Aprenderás sobre bloques, extensiones, segmentos y espacios de tablas, así como el uso de bitácoras para registrar cambios mediante triggers aplicados a inserciones, actualizaciones y eliminaciones. También se abordan las particiones, una técnica clave para mejorar el rendimiento y la administración de datos dividiendo tablas e índices en partes más manejables. Se explican distintos tipos de particionado (horizontal, vertical, por rango, lista, hash, clave y combinados), con ejemplos prácticos. Por último, conocerás los espacios privados, interfaces personalizadas como paneles de control que facilitan el uso, aumentan la seguridad y mejoran la organización de la base de datos para distintos tipos de usuarios.

Sección 1: Definición de espacio de almacenamiento.

En la gestión del almacenamiento de una base de datos, es fundamental comprender cómo se organizan físicamente los datos en el sistema. Para ello, existen cuatro conceptos clave que describen la jerarquía y la estructura de almacenamiento en un SGBD, especialmente en sistemas como Oracle: **bloque, extensión, segmento y espacio de tablas (tablespace)**. Estos conceptos constituyen las estructuras lógicas de almacenamiento que permiten una gestión eficiente de los datos, abstraídos del hardware subyacente.

Bloque (Block)

El bloque es la unidad más pequeña de almacenamiento que maneja una base de datos. Se puede entender como un contenedor de datos que contiene una cantidad fija de bytes, y representa la mínima unidad de E/S (entrada/salida) entre la base de datos y el sistema operativo.

- **Importancia:** Es crucial que el tamaño del bloque de la base de datos sea un múltiplo del tamaño del bloque del sistema operativo. De lo contrario, se incurre en una penalización de rendimiento, ya que se requerirá trabajo adicional para realizar operaciones de lectura y escritura.
- **Parámetro relevante:** En Oracle, por ejemplo, se especifica mediante el parámetro `DB_BLOCK_SIZE`.



- **Contenido:** Cada bloque puede contener una o varias filas de una tabla o parte de una fila muy grande (por ejemplo, objetos LOB). **Extensiones:** Se forma con uno o más bloques. Cuando se aumenta el tamaño de un objeto se usa una extensión para incrementar el espacio.

Extensiones (Extents)

Una extensión es un conjunto contiguo de bloques que el sistema gestiona como una unidad para fines de asignación de espacio. Cuando una tabla o un índice necesita más espacio, el sistema le asigna una nueva extensión.

- **Ventajas:** Las extensiones permiten una asignación progresiva del espacio, lo que evita el desperdicio al no tener que reservar grandes cantidades de espacio de una sola vez.
- **Relación:** Un segmento está compuesto por una o varias extensiones.

Segmentos (Segments)

Los segmentos son estructuras lógicas más grandes que están formadas por múltiples extensiones. Cada objeto de base de datos que almacena datos persistentemente como una tabla, un índice, una vista materializada o un clúster posee su propio segmento.

- **Tipos:** Existen varios tipos de segmentos, entre ellos:
 - Segmentos de tabla: para almacenar datos de tablas.
 - Segmentos de índice: para almacenar estructuras de índice.
 - Segmentos temporales: utilizados por operaciones que requieren espacio temporal.
 - Segmentos de undo: usados para el control de transacciones y recuperación.
- **Gestión:** La forma en que se asignan y liberan extensiones dentro de un segmento afecta directamente al rendimiento del sistema.

Espacio de Tablas (Tablespace)

El tablespace es la unidad lógica más grande de almacenamiento. Consiste en uno o varios archivos físicos llamados datafiles y agrupa a varios segmentos.

- **Características:**
 - Cada datafile pertenece a un único tablespace.
 - Un tablespace puede contener múltiples objetos de diferentes esquemas (usuarios).
 - Pueden definirse tablespaces especializados, como:
 - SYSAUX: para almacenamiento auxiliar del sistema.
 - TEMP: para operaciones temporales.
 - UNDO: para manejo de transacciones.
- **Importancia:** Permiten una administración flexible de los recursos, controlando la ubicación física y lógica de los datos.

Consideraciones Generales sobre el Almacenamiento en Bases de Datos

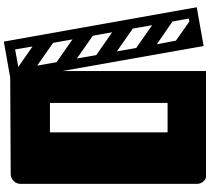
El almacenamiento de los objetos de la base de datos (tablas, índices, vistas materializadas, etc.) no se realiza directamente sobre los archivos físicos, sino a través de estas estructuras lógicas de almacenamiento. Esta separación permite la portabilidad, independencia del hardware y flexibilidad en la administración de recursos.



Las bases de datos están diseñadas para almacenar grandes volúmenes de datos de manera permanente y confiable. Generalmente, estos datos se almacenan en memorias secundarias (como discos duros o SSDs), por varias razones:

- Las bases de datos suelen exceder la capacidad de la memoria primaria (RAM).
- Las memorias secundarias son más económicas, aunque más lentas.
- A diferencia de la memoria primaria, las memorias secundarias no son volátiles: conservan los datos incluso después de apagar el sistema.

Ocasionalmente, también se utilizan dispositivos como discos ópticos o memorias flash, especialmente en entornos con requerimientos de portabilidad o alta disponibilidad.



Actividad Interactiva #1

Mira el video del link presentado a continuación y escribe en las líneas de abajo un resumen de 4 líneas de lo que consideres más importante.

<https://www.youtube.com/watch?v=iwwOyvYgYiM>

Sección 2: Definición y creación del espacio asignado para cada base de datos.

Cuando se crea una base de datos, el DBMS (Sistema de Gestión de Bases de Datos) debe asignar un espacio físico para almacenar los datos. Este espacio se asigna a través de archivos y puede organizarse de diferentes maneras, dependiendo de la estrategia de acceso y de la estructura lógica utilizada.

Organización Primaria de Archivos

Las formas más comunes de organizar físicamente los archivos de una base de datos en disco son:

- Archivos de Montículos (Heap files o No Ordenados)
 - Los registros se almacenan en el orden en que se insertan.
 - No hay ningún criterio de orden predefinido.
 - Ventajas: Simplicidad y rapidez en inserciones.
 - Desventajas: Ineficiente para búsquedas específicas.
- Archivos Ordenados (Sequential files):
 - Los registros se almacenan siguiendo un orden basado en un campo clave.
 - Ventajas: Rápidas búsquedas por rango y eficiencia en ordenamientos.
 - Desventajas: Las inserciones y eliminaciones son más costosas.
- Archivos con Direccionamiento Calculado (Hashing):
 - Se aplica una función hash sobre un campo clave para determinar la ubicación del registro.
 - Ventajas: Búsqueda directa muy rápida.
 - Desventajas: Colisiones y dificultad para búsquedas por rango.
- Árboles B y B+ (B-tree structures):
 - Se utilizan estructuras balanceadas de árboles para organizar los datos.
 - Muy eficientes para operaciones de búsqueda, inserción y eliminación.



Organización Secundaria de Archivos (Índices Auxiliares)

Además de la organización primaria, se pueden utilizar estructuras de acceso secundario (índices), que permiten el acceso eficiente a los registros basados en campos alternativos. Estos índices pueden ser:

- Índices simples: sobre un campo que no es clave primaria.
- Índices compuestos: sobre varios campos.
- Índices únicos: garantizan que no haya duplicados.
- Índices bitmaps, hash, entre otros.

Asignación de Espacio en el DBMS

El espacio de almacenamiento se asigna a través de comandos como CREATE DATABASE o ALTER DATABASE. Estos comandos permiten especificar:

- El nombre de los dispositivos o archivos donde se almacenará la base de datos.
- El tamaño que se asignará a cada dispositivo.
- El uso de ubicaciones predeterminadas o personalizadas.

Sintaxis y Ejemplos de Asignación

1. Asignación en ubicación predeterminada:

```
CREATE DATABASE newpubs ON DEFAULT = 4;
```

- Crea una base de datos llamada newpubs con 4MB de espacio en el dispositivo predeterminado.

2. Asignación en dispositivos específicos:

```
CREATE DATABASE newdb ON mydata = 3, newdata = 2;
```

- Crea una base de datos llamada newdb con 3MB en el dispositivo mydata y 2MB en newdata.
- Nota Importante: Siempre es recomendable almacenar el log de transacciones (transaction log) en un dispositivo diferente al de los datos, para mejorar la integridad y el rendimiento.
-

3. Inicialización de Dispositivos (disk init):

- Antes de asignar espacio a un dispositivo específico, este debe estar registrado en sysdevices.
- Esto se logra utilizando el comando disk init.

Comportamiento en Caso de Espacio Insuficiente

- Si el dispositivo especificado no tiene el espacio suficiente, el DBMS intentará asignar tanto como pueda, notificando al usuario.
- Si el espacio es inferior al mínimo requerido, el comando CREATE DATABASE fallará.

La definición y gestión del espacio de almacenamiento son aspectos fundamentales en la administración de bases de datos. Comprender las estructuras lógicas como bloques, extensiones, segmentos y tablespaces permite optimizar el uso de recursos y garantizar el rendimiento del sistema.



Sección 3: Bitácoras.

La bitácora, también conocida como log de transacciones, es una estructura fundamental en la gestión de bases de datos modernas. Su objetivo principal es registrar todas las operaciones de modificación realizadas sobre los datos. Sirve tanto para propósitos de auditoría, control de cambios, recuperación ante fallos y mantenimiento de la integridad de los datos.

El sistema de bitácora es una herramienta esencial en los mecanismos de recuperación ante fallos (recovery), ya que permite deshacer (undo) o rehacer (redo) transacciones en función de su estado (commit o rollback).

Estructura de un Registro de Bitácora

Cada operación de escritura que se realiza sobre una base de datos debe generar un registro en la bitácora antes de aplicar el cambio a los datos. Este principio se conoce como "Write-Ahead Logging" (WAL) y garantiza que los cambios puedan recuperarse incluso si ocurre una falla en el sistema.

Cada registro típico de bitácora contiene:

1. Nombre de la transacción (T): Identificador único de la transacción que realiza la operación.
2. Nombre del dato (X): Nombre o identificador del campo, columna o variable afectada.
3. Valor antiguo (v1): El valor del dato antes de realizar la operación.
4. Valor nuevo (v2): El valor del dato después de la operación.

Ejemplo:

< T1, inicio >

< T1, x, 30, 45 >

< T1, commit >

Donde:

- T1 inicia una transacción.
- Modifica el valor de x de 30 a 45.
- La transacción finaliza correctamente con un commit.

Este modelo asegura que, en caso de una falla, si una transacción no fue comprometida (no hizo commit), sus efectos se pueden deshacer utilizando el valor antiguo. Por el contrario, si sí se comprometió, se pueden rehacer usando el valor nuevo.

Importancia del Registro de Bitácora

Los registros de la bitácora son clave en la implementación de consistencia y durabilidad, dos propiedades esenciales del modelo ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad) que define el comportamiento esperado de las transacciones en bases de datos.



Ventajas:

- Permite deshacer cambios indebidos.
- Facilita auditorías de seguridad y cumplimiento.
- Mejora el mantenimiento y depuración del sistema.
- Es esencial para la recuperación tras fallas del sistema.

Dónde se Almacenan los Registros de Bitácora

Para garantizar su confiabilidad, los registros de bitácora deben almacenarse en una memoria estable (disco duro, SSD o medios persistentes). No deben depender de la memoria principal (RAM), ya que esta es volátil y se borra en caso de fallas del sistema.

Ejemplo Practico: Bitácora en PostgreSQL

Para ilustrar la aplicación práctica de la bitácora en PostgreSQL, supongamos que trabajamos con una base de datos llamada proyecto que contiene tres tablas: carrera, departamento y maestros.

Creación de la base de datos y definición de las tablas:

```
CREATE DATABASE proyecto;
```

```
CREATE TABLE carrera (  
    clave_carrera INT PRIMARY KEY,  
    nom_carrera VARCHAR(20) NOT NULL,  
    num_depto INT NOT NULL  
);
```

```
CREATE TABLE departamento (  
    num_departamento INT PRIMARY KEY,  
    nombre_dept VARCHAR(20) NOT NULL,  
    jefe_num_tarjet INT NOT NULL  
);
```

```
CREATE TABLE maestros (  
    num_tarjeta INT PRIMARY KEY,  
    nombre VARCHAR(50)  
);
```

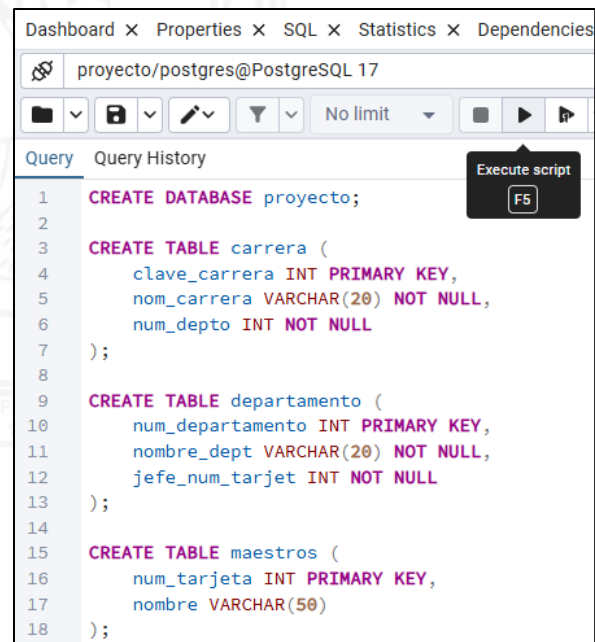


Ilustración 1 Definición de la tablas en pgAdmin4.



Tabla de Bitácora:

```
CREATE TABLE bitacora (  
    id SERIAL PRIMARY KEY,  
    operacion VARCHAR(10),  
    usuario VARCHAR(40),  
    host VARCHAR(30) NOT NULL,  
    modificado TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    tabla VARCHAR(40) NOT NULL  
);
```

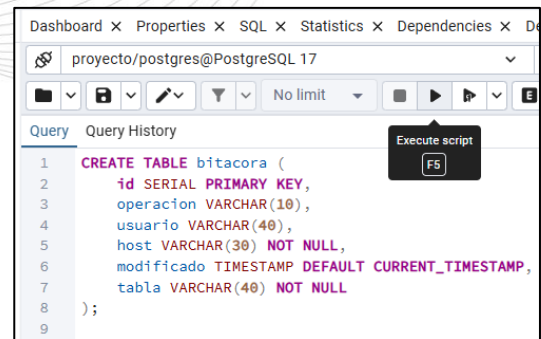


Ilustración 2 Definición de la tabla bitácora en pgAdmin4.

La tabla bitacora registrará automáticamente las operaciones realizadas (INSERT, UPDATE, DELETE), quién las hizo (usuario), desde qué host, en qué fecha/hora y en qué tabla.

Función para insertar en la bitácora

En PostgreSQL, los triggers llaman funciones.

Esta es la función que usaremos para la tabla carrera:

```
CREATE OR REPLACE FUNCTION registrar_bitacora_carrera()  
RETURNS TRIGGER AS $$  
BEGIN  
    INSERT INTO bitacora (host, usuario, operacion, modificado, tabla)  
    VALUES (  
        inet_client_addr(),  
        current_user,  
        TG_OP,  
        CURRENT_TIMESTAMP,  
        'CARRERA'  
    );  
    RETURN NULL;  
END;  
$$ LANGUAGE plpgsql;
```

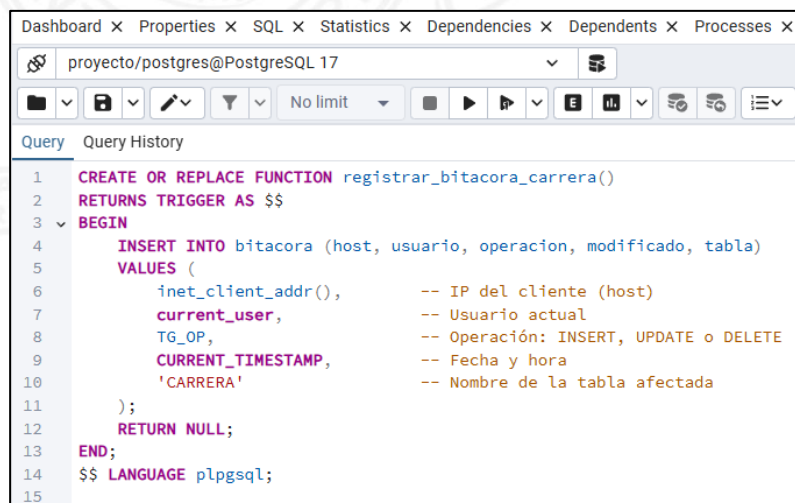


Ilustración 3 Función para el trigger de la tabla carrera en pgAdmin4



Esta es la función que usaremos para la tabla departamento:

```
CREATE OR REPLACE FUNCTION registrar_bitacora_departamento()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
INSERT INTO bitacora (host, usuario, operacion, modificado, tabla)
```

```
VALUES (
```

```
inet_client_addr(),
```

```
current_user,
```

```
TG_OP,
```

```
CURRENT_TIMESTAMP,
```

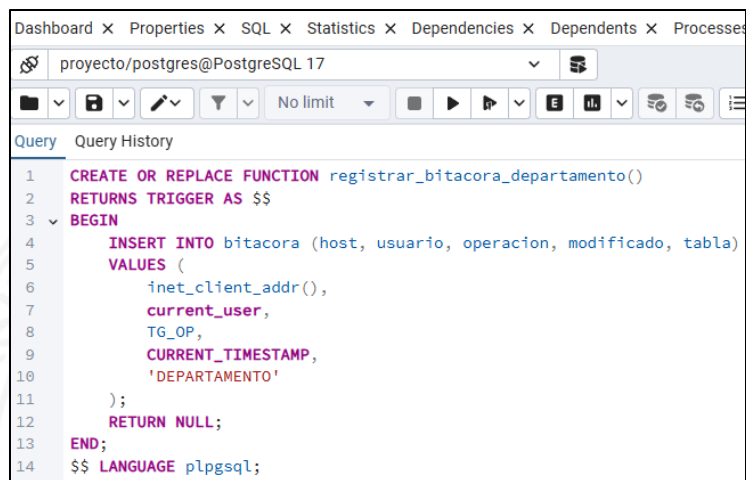
```
'DEPARTAMENTO'
```

```
);
```

```
RETURN NULL;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

A screenshot of the pgAdmin 4 interface. The top navigation bar includes tabs for Dashboard, Properties, SQL, Statistics, Dependencies, Dependents, and Processes. The main window shows a query editor with the following SQL code:

```
1 CREATE OR REPLACE FUNCTION registrar_bitacora_departamento()  
2 RETURNS TRIGGER AS $$  
3 BEGIN  
4     INSERT INTO bitacora (host, usuario, operacion, modificado, tabla)  
5     VALUES (  
6         inet_client_addr(),  
7         current_user,  
8         TG_OP,  
9         CURRENT_TIMESTAMP,  
10        'DEPARTAMENTO'  
11    );  
12    RETURN NULL;  
13 END;  
14 $$ LANGUAGE plpgsql;
```

Ilustración 4 Función para el trigger de la tabla departamento en pgAdmin4

Esta es la función genérica usaremos para la tabla maestros:

```
CREATE OR REPLACE FUNCTION registrar_bitacora_maestros()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
INSERT INTO bitacora (host, usuario, operacion, modificado, tabla)
```

```
VALUES (
```

```
inet_client_addr(),
```

```
current_user,
```

```
TG_OP,
```

```
CURRENT_TIMESTAMP,
```

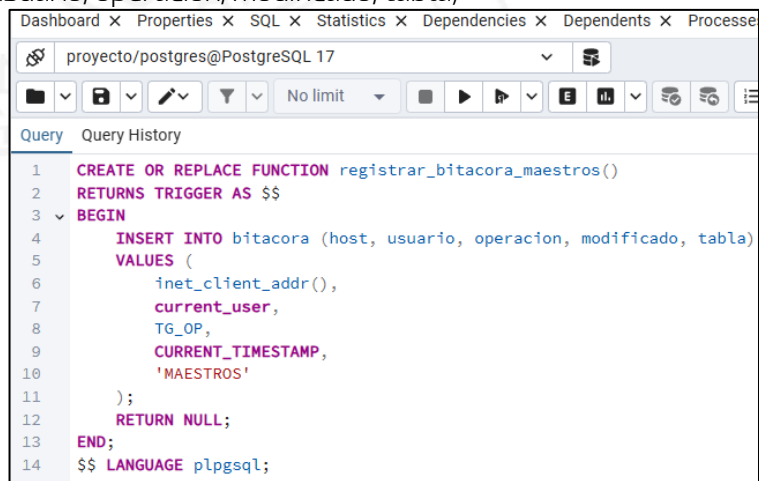
```
'MAESTROS'
```

```
);
```

```
RETURN NULL;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

A screenshot of the pgAdmin 4 interface, similar to the previous one. The main window shows a query editor with the following SQL code:

```
1 CREATE OR REPLACE FUNCTION registrar_bitacora_maestros()  
2 RETURNS TRIGGER AS $$  
3 BEGIN  
4     INSERT INTO bitacora (host, usuario, operacion, modificado, tabla)  
5     VALUES (  
6         inet_client_addr(),  
7         current_user,  
8         TG_OP,  
9         CURRENT_TIMESTAMP,  
10        'MAESTROS'  
11    );  
12    RETURN NULL;  
13 END;  
14 $$ LANGUAGE plpgsql;
```

Ilustración 5 Función para el trigger de la tabla maestros en pgAdmin4



Triggers para Automatizar la Bitácora

Para garantizar que las operaciones de insertar, eliminar y actualizar se registren automáticamente, se utilizan triggers (disparadores) en cada tabla. Cada tabla necesita 3 triggers, lo que da un total de 9 triggers para registrar las operaciones de las 3 tablas.

Triggers para la tabla carrera:

-- Trigger AFTER INSERT

CREATE TRIGGER bit_carr_ins

AFTER INSERT ON carrera

FOR EACH ROW

EXECUTE FUNCTION registrar_bitacora_carrera();

-- Trigger AFTER UPDATE

CREATE TRIGGER bit_carr_upd

AFTER UPDATE ON carrera

FOR EACH ROW

EXECUTE FUNCTION registrar_bitacora_carrera();

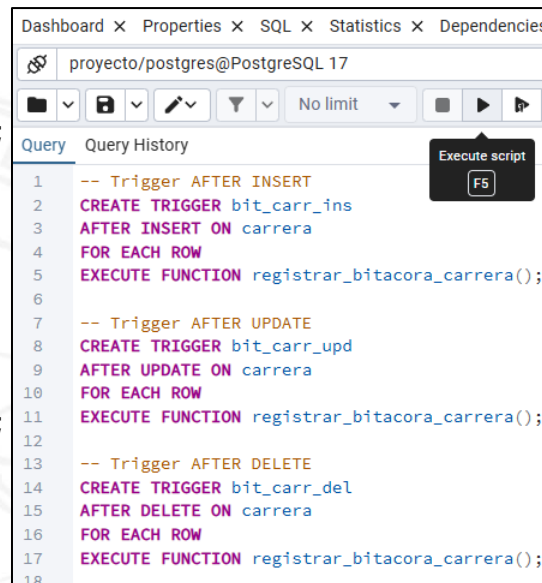
-- Trigger AFTER DELETE

CREATE TRIGGER bit_carr_del

AFTER DELETE ON carrera

FOR EACH ROW

EXECUTE FUNCTION registrar_bitacora_carrera();



```
1  -- Trigger AFTER INSERT
2  CREATE TRIGGER bit_carr_ins
3  AFTER INSERT ON carrera
4  FOR EACH ROW
5  EXECUTE FUNCTION registrar_bitacora_carrera();
6
7  -- Trigger AFTER UPDATE
8  CREATE TRIGGER bit_carr_upd
9  AFTER UPDATE ON carrera
10 FOR EACH ROW
11 EXECUTE FUNCTION registrar_bitacora_carrera();
12
13 -- Trigger AFTER DELETE
14 CREATE TRIGGER bit_carr_del
15 AFTER DELETE ON carrera
16 FOR EACH ROW
17 EXECUTE FUNCTION registrar_bitacora_carrera();
18
```

Ilustración 6 Triggers correspondientes a la tabla carrera en pgAdmin4

Triggers para la tabla departamento:

-- AFTER INSERT

CREATE TRIGGER bit_depto_ins

AFTER INSERT ON departamento

FOR EACH ROW

EXECUTE FUNCTION registrar_bitacora_departamento();

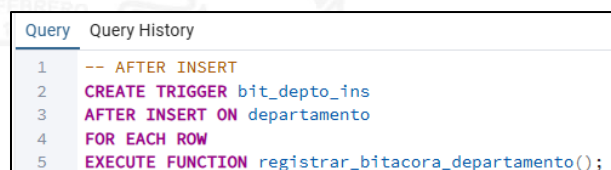
-- AFTER UPDATE

CREATE TRIGGER bit_depto_upd

AFTER UPDATE ON departamento

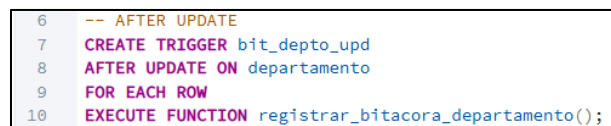
FOR EACH ROW

EXECUTE FUNCTION registrar_bitacora_departamento();



```
1  -- AFTER INSERT
2  CREATE TRIGGER bit_depto_ins
3  AFTER INSERT ON departamento
4  FOR EACH ROW
5  EXECUTE FUNCTION registrar_bitacora_departamento();
```

Ilustración 7 Trigger after insert de la tabla carrera en pgAdmin4



```
6  -- AFTER UPDATE
7  CREATE TRIGGER bit_depto_upd
8  AFTER UPDATE ON departamento
9  FOR EACH ROW
10 EXECUTE FUNCTION registrar_bitacora_departamento();
```

Ilustración 8 Trigger after update de la tabla carrera en pgAdmin4



-- AFTER DELETE

```
CREATE TRIGGER bit_depto_del  
AFTER DELETE ON departamento  
FOR EACH ROW
```

```
EXECUTE FUNCTION registrar_bitacora_departamento();
```

```
11 -- AFTER DELETE  
12 CREATE TRIGGER bit_depto_del  
13 AFTER DELETE ON departamento  
14 FOR EACH ROW  
15 EXECUTE FUNCTION registrar_bitacora_departamento();
```

Ilustración 9 Trigger delete insert de la tabla carrera en pgAdmin4

Triggers para la tabla maestros:

-- AFTER INSERT

```
CREATE TRIGGER bit_maestros_ins  
AFTER INSERT ON maestros  
FOR EACH ROW
```

```
EXECUTE FUNCTION registrar_bitacora_maestros();
```

-- AFTER UPDATE

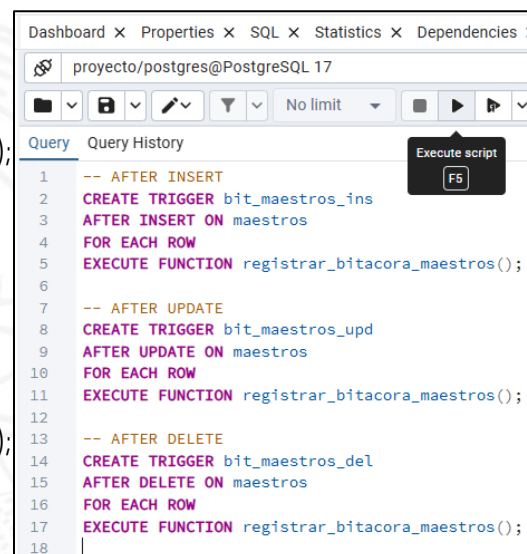
```
CREATE TRIGGER bit_maestros_upd  
AFTER UPDATE ON maestros  
FOR EACH ROW
```

```
EXECUTE FUNCTION registrar_bitacora_maestros();
```

-- AFTER DELETE

```
CREATE TRIGGER bit_maestros_del  
AFTER DELETE ON maestros  
FOR EACH ROW
```

```
EXECUTE FUNCTION registrar_bitacora_maestros();
```



The screenshot shows the pgAdmin4 SQL editor interface. The top bar includes tabs for Dashboard, Properties, SQL, Statistics, and Dependencies. The main window displays a SQL query for creating three triggers on the 'maestros' table: an AFTER INSERT trigger, an AFTER UPDATE trigger, and an AFTER DELETE trigger. Each trigger calls the 'registrar_bitacora_maestros()' function. The interface includes a toolbar with icons for saving, running, and other database actions, and a 'Query History' panel on the right.

Ilustración 10 Triggers correspondientes a la tabla maestros en pgAdmin4

Estos triggers haciendo uso de su respectiva función capturan automáticamente el usuario, el host, la operación realizada y la tabla modificada, garantizando un registro completo de las operaciones.

Insertar registros de prueba

Tabla carrera:

```
INSERT INTO carrera (clave_carrera, nom_carrera, num_depto)  
VALUES (1, 'Ingeniería', 10);
```

Actualizar registros

```
UPDATE carrera SET nom_carrera = 'Ing. Software' WHERE clave_carrera = 1;
```



Eliminar registros

DELETE FROM carrera WHERE clave_carrera = 1;

Consultar la bitácora:

SELECT * FROM bitacora ORDER BY modificado DESC;



Ilustración 11 Consulta de la bitácora en pgAdmin4

Esto te mostrará lo siguiente:

The screenshot shows the 'Data Output' tab in pgAdmin4. It displays the results of the query 'SELECT * FROM bitacora ORDER BY modificado DESC;'. The results are shown in a table with the following columns: id, operacion, usuario, host, modificado, and tabla.

	id [PK] integer	operacion character varying (10)	usuario character varying (40)	host character varying (30)	modificado timestamp without time zone	tabla character varying (40)
1	3	DELETE	postgres	::1/128	2025-08-24 22:26:30.863173	CARRERA
2	2	UPDATE	postgres	::1/128	2025-08-24 22:25:03.820927	CARRERA
3	1	INSERT	postgres	::1/128	2025-08-24 22:24:17.157544	CARRERA

Ilustración 12 Resultado de la consulta de la bitácora

Ejemplo Practico: Bitácora en MySQL

Supongamos que trabajamos con una base de datos llamada proyecto que contiene tres tablas: carrera, departamento y maestros.

Creación de la base de datos y definición de las tablas:

CREATE DATABASE proyecto;

USE proyecto;

CREATE TABLE carrera (

clave_carrera INT PRIMARY KEY,
nom_carrera VARCHAR(20) NOT NULL,
num_depto INT NOT NULL

);

CREATE TABLE departamento (

num_departamento INT PRIMARY KEY,
nombre_dept VARCHAR(20) NOT NULL,
jefe_num_tarjet INT NOT NULL

);

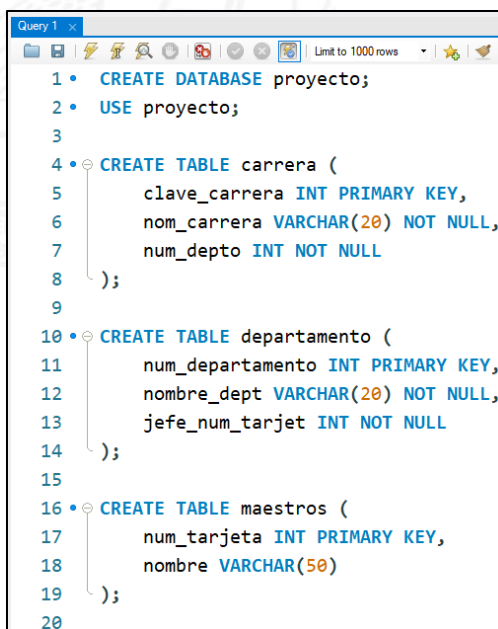


Ilustración 13 Definición de las tablas en MySQL Workbench.



```
CREATE TABLE maestros (
    num_tarjeta INT PRIMARY KEY,
    nombre VARCHAR(50)
);
```

Tabla de Bitácora:

```
CREATE TABLE bitacora (
    id INT AUTO_INCREMENT PRIMARY KEY,
    operacion VARCHAR(10),
    usuario VARCHAR(40),
    host VARCHAR(30) NOT NULL,
    modificado TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    tabla VARCHAR(40) NOT NULL
);
```

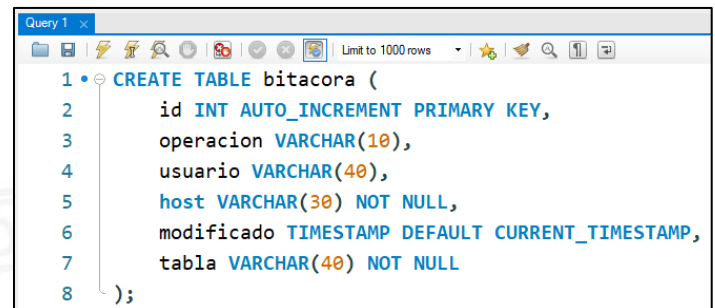


Ilustración 14 Definición de la tabla bitácora en MySQL Workbench.

La tabla bitacora registrará automáticamente las operaciones realizadas (INSERT, UPDATE, DELETE), quién las hizo (usuario), desde qué host, en qué fecha/hora y en qué tabla.

Crear triggers para registrar las operaciones

Nota importante: En MySQL, los triggers deben contener el código directamente, ya que no se pueden enlazar con funciones como en PostgreSQL.

Triggers para las tablas carrera, departamento y maestros.

```
DELIMITER //

-- Triggers para la tabla carrera

CREATE TRIGGER bit_carr_ins AFTER INSERT ON carrera
FOR EACH ROW
BEGIN
    INSERT INTO bitacora (host, usuario, operacion, modificado, tabla)
    VALUES (
        SUBSTRING_INDEX(USER(), '@', -1),
        SUBSTRING_INDEX(USER(), '@', 1),
        'INSERT',
        CURRENT_TIMESTAMP,
        'CARRERA'
    );
END//
```



```

CREATE TRIGGER bit_carr_upd AFTER UPDATE ON carrera
FOR EACH ROW
BEGIN
    INSERT INTO bitacora (host, usuario, operacion, modificado, tabla)
    VALUES (
        SUBSTRING_INDEX(USER(), '@', -1),
        SUBSTRING_INDEX(USER(), '@', 1),
        'UPDATE',
        CURRENT_TIMESTAMP,
        'CARRERA'
    );
END//

```

```

CREATE TRIGGER bit_carr_del AFTER DELETE ON carrera
FOR EACH ROW
BEGIN
    INSERT INTO bitacora (host, usuario, operacion, modificado, tabla)
    VALUES (
        SUBSTRING_INDEX(USER(), '@', -1),
        SUBSTRING_INDEX(USER(), '@', 1),
        'DELETE',
        CURRENT_TIMESTAMP,
        'CARRERA'
    );
END//

```

-- Triggers para la tabla departamento

```

CREATE TRIGGER bit_depto_ins AFTER INSERT ON departamento
FOR EACH ROW
BEGIN
    INSERT INTO bitacora (host, usuario, operacion, modificado, tabla)
    VALUES (
        SUBSTRING_INDEX(USER(), '@', -1),
        SUBSTRING_INDEX(USER(), '@', 1),
        'INSERT',
        CURRENT_TIMESTAMP,
        'DEPARTAMENTO'
    );
END//

```

```

CREATE TRIGGER bit_depto_upd AFTER UPDATE ON departamento
FOR EACH ROW
BEGIN
    INSERT INTO bitacora (host, usuario, operacion, modificado, tabla)
    VALUES (

```



```
SUBSTRING_INDEX(USER(), '@', -1),
SUBSTRING_INDEX(USER(), '@', 1),
'UPDATE',
CURRENT_TIMESTAMP,
'DEPARTAMENTO'
);
END//
```

```
CREATE TRIGGER bit_depto_del AFTER DELETE ON departamento
FOR EACH ROW
BEGIN
  INSERT INTO bitacora (host, usuario, operacion, modificado, tabla)
  VALUES (
    SUBSTRING_INDEX(USER(), '@', -1),
    SUBSTRING_INDEX(USER(), '@', 1),
    'DELETE',
    CURRENT_TIMESTAMP,
    'DEPARTAMENTO'
  );
END//
```

-- Triggers para la tabla maestros

```
CREATE TRIGGER bit_maestros_ins AFTER INSERT ON maestros
FOR EACH ROW
BEGIN
  INSERT INTO bitacora (host, usuario, operacion, modificado, tabla)
  VALUES (
    SUBSTRING_INDEX(USER(), '@', -1),
    SUBSTRING_INDEX(USER(), '@', 1),
    'INSERT',
    CURRENT_TIMESTAMP,
    'MAESTROS'
  );
END//
```

```
CREATE TRIGGER bit_maestros_upd AFTER UPDATE ON maestros
FOR EACH ROW
BEGIN
  INSERT INTO bitacora (host, usuario, operacion, modificado, tabla)
  VALUES (
    SUBSTRING_INDEX(USER(), '@', -1),
    SUBSTRING_INDEX(USER(), '@', 1),
    'UPDATE',
    CURRENT_TIMESTAMP,
    'MAESTROS'
  );
END//
```



END//

```
CREATE TRIGGER bit_maestros_del AFTER DELETE ON maestros
FOR EACH ROW
BEGIN
    INSERT INTO bitacora (host, usuario, operacion, modificado, tabla)
    VALUES (
        SUBSTRING_INDEX(USER(), '@', -1),
        SUBSTRING_INDEX(USER(), '@', 1),
        'DELETE',
        CURRENT_TIMESTAMP,
        'MAESTROS'
    );
END//

DELIMITER ;
```

Insertar registros de prueba

Tabla carrera:

```
INSERT INTO carrera (clave_carrera, nom_carrera, num_depto)
VALUES (1, 'Ingeniería', 10);
```

Actualizar registros

```
UPDATE carrera SET nom_carrera = 'Ing. Software' WHERE clave_carrera = 1;
```

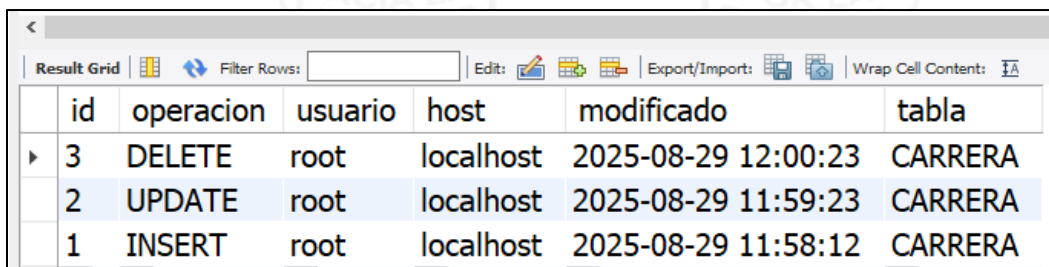
Eliminar registros

```
DELETE FROM carrera WHERE clave_carrera = 1;
```

Consultar la bitácora:

```
SELECT * FROM bitacora ORDER BY modificado DESC;
```

Esto te mostrará lo siguiente:



	id	operacion	usuario	host	modificado	tabla
▶	3	DELETE	root	localhost	2025-08-29 12:00:23	CARRERA
	2	UPDATE	root	localhost	2025-08-29 11:59:23	CARRERA
	1	INSERT	root	localhost	2025-08-29 11:58:12	CARRERA

Ilustración 15 Resultado de la consulta de la bitácora



Actividad Interactiva #2

Ejercicio de asociación.

A continuación, verás una lista de términos y una lista de definiciones. Asocia Cada Término con su Definición: Une cada término con la definición correcta.

Términos:

1. Bitácora
2. Bloque
3. Extensión
4. Segmento
5. Tablespace
6. Trigger



Definiciones:

- A. Estructura lógica de almacenamiento compuesta por varias extensiones; agrupa los datos de un objeto como una tabla o un índice.
- B. Estructura lógica más grande que organiza y agrupa segmentos; está asociada a uno o más archivos físicos llamados datafiles.
- C. Registro que almacena detalles sobre cada operación de modificación en una base de datos, incluyendo usuario, operación, tabla, fecha, entre otros.
- D. Unidad mínima de almacenamiento gestionada por el DBMS; suele ser múltiplo del tamaño del bloque del sistema operativo.
- E. Conjunto de bloques contiguos asignados como unidad de espacio para expandir objetos de base de dato.
- F. Mecanismo automático que se ejecuta tras una operación INSERT, UPDATE o DELETE, permitiendo ejecutar acciones como registrar cambios en una bitácora

Instrucciones: Asocia cada término (1-6) con la definición correspondiente (A - F). Y escríbela a continuación.
Ej: "1 - C"

Sección 4: Particiones.

En el contexto de bases de datos, una partición se refiere a la división de una base de datos lógica o de sus elementos constituyentes (como tablas, índices o registros) en partes más pequeñas e independientes. Esta técnica se utiliza comúnmente con el objetivo de mejorar el rendimiento, facilitar el mantenimiento y optimizar la administración de los datos.

Al implementar particiones, es posible distribuir los datos en múltiples nodos o servidores. De esta manera, los usuarios en un nodo específico pueden realizar transacciones locales en su propia partición, sin necesidad de acceder al conjunto completo de datos. Esto no solo mejora el tiempo de respuesta, sino que también incrementa la disponibilidad del sistema, ya que una falla en una partición no afecta directamente a las demás.

Otra ventaja relevante de la partición es la seguridad, ya que puede restringirse el acceso a ciertas particiones basándose en los roles de usuario. Además, las particiones permiten realizar tareas de mantenimiento sin interrumpir todo el sistema, ya que las operaciones pueden ejecutarse de forma aislada en una sola partición.

La partición puede lograrse de dos formas generales:

- **Dividiendo la base de datos en bases de datos más pequeñas e independientes**, donde cada una cuenta con sus propias tablas, índices y registros de transacción.



- **Dividiendo elementos específicos de la base de datos**, como una tabla concreta que contenga gran cantidad de datos.

Tipos de partición

Existen principalmente dos formas de particionar una tabla: **partición horizontal y partición vertical**.

Partición Horizontal

La partición horizontal divide una tabla en varias subtablas que contienen diferentes filas, pero todas las subtablas comparten la misma estructura de columnas. Esta estrategia se usa comúnmente cuando diferentes grupos de datos son consultados con frecuencia de manera independiente.

Ejemplo práctico: Supongamos que tenemos una tabla llamada Clientes que almacena información de todos los clientes de una empresa. Podríamos dividir esta tabla en dos particiones basadas en el código postal:

Clientes con código postal menor a 50000 = ClientesEste

Clientes con código postal igual o mayor a 50000 = ClientesOeste

Ambas tablas (ClientesEste y ClientesOeste) contienen las mismas columnas, pero diferentes conjuntos de filas. Para ofrecer una visión unificada, puede crearse una vista que combine ambas tablas mediante una cláusula UNION.

Partición Vertical

La partición vertical consiste en dividir una tabla por columnas, es decir, se separan grupos de columnas en distintas tablas, generalmente dejando la clave primaria o algún identificador común en cada una de ellas para que puedan relacionarse entre sí.

Esto se utiliza cuando una tabla contiene una gran cantidad de columnas y algunas de ellas no se utilizan con frecuencia. Separando las columnas menos utilizadas, se reduce la carga de las consultas comunes.

Métodos de Partición

1. Partición por Rango

En esta técnica, los datos se distribuyen en particiones en función de un rango continuo de valores. Es útil cuando los datos tienen un orden natural, como fechas o cantidades numéricas.

Ejemplo: Queremos particionar una tabla contratos basada en el año de inicio del contrato (fecha_inicio):

```
CREATE TABLE contratos (
```

```
    id SERIAL,
```

```
    cliente TEXT,
```

```
    fecha_inicio DATE
```



) PARTITION BY RANGE (fecha_inicio);

```
CREATE TABLE contratos_1950_1960 PARTITION OF contratos  
FOR VALUES FROM ('1950-01-01') TO ('1960-01-01');
```

```
CREATE TABLE contratos_1960_1970 PARTITION OF contratos  
FOR VALUES FROM ('1960-01-01') TO ('1970-01-01');
```

```
CREATE TABLE contratos_1970_1980 PARTITION OF contratos  
FOR VALUES FROM ('1970-01-01') TO ('1980-01-01');
```

Cada tabla hija (contratos_1950_1960, etc.) almacenará solo las filas que correspondan al rango especificado.

2. Partición por Lista

Aquí se asignan filas a particiones según un conjunto específico de valores predefinidos. Es útil cuando los datos tienen un conjunto discreto de valores posibles.

Ejemplo:

```
CREATE TABLE ventas (  
    id SERIAL,  
    region TEXT,  
    total NUMERIC
```

```
) PARTITION BY LIST (region);
```

```
CREATE TABLE ventas_norte PARTITION OF ventas  
FOR VALUES IN ('Norte');
```

```
CREATE TABLE ventas_sur PARTITION OF ventas  
FOR VALUES IN ('Sur');
```

```
CREATE TABLE ventas_centro PARTITION OF ventas  
FOR VALUES IN ('Centro');
```

Cada tabla hija (ventas_norte, ventas_sur, ventas_centro) almacenará solo las filas que correspondan al valor específico de región indicado ('Norte', 'Sur', 'Centro').

3. Partición por Hash

En este método, se utiliza una función de hash interna para distribuir filas de manera equitativa entre particiones. Es ideal para balances de carga donde no hay un rango claro o valores discretos conocidos.



Ejemplo:

```
CREATE TABLE logs (
```

```
    id SERIAL,
```

```
    mensaje TEXT
```

```
) PARTITION BY HASH (id);
```

```
CREATE TABLE logs_part_0 PARTITION OF logs FOR VALUES WITH (MODULUS 4,  
REMAINDER 0);
```

```
CREATE TABLE logs_part_1 PARTITION OF logs FOR VALUES WITH (MODULUS 4,  
REMAINDER 1);
```

```
CREATE TABLE logs_part_2 PARTITION OF logs FOR VALUES WITH (MODULUS 4,  
REMAINDER 2);
```

```
CREATE TABLE logs_part_3 PARTITION OF logs FOR VALUES WITH (MODULUS 4,  
REMAINDER 3);
```

Esto distribuye automáticamente los registros entre 4 particiones, logrando un acceso más eficiente en sistemas con mucha concurrencia. Esto es útil cuando se requiere una distribución uniforme de los datos, especialmente si no hay un criterio claro de particionado.

4. Partición por Clave (Key)

Es similar a la partición por hash, pero en este caso MySQL usa una función interna de hash y puede trabajar directamente con columnas no numéricas. Si no se especifica ninguna columna, se utiliza la clave primaria por defecto.

Ejemplo:

```
ALTER TABLE contratos
```

```
PARTITION BY KEY()
```

```
PARTITIONS 7;
```

La partición por clave (KEY) es una característica de MySQL, no de PostgreSQL.

5. Particiones Compuestas

Este enfoque permite combinar varios tipos de partición. Por ejemplo, se puede usar primero una partición por rango, y dentro de cada rango, una partición adicional por hash. Esto proporciona un control más granular sobre la distribución y acceso a los datos.

PostgreSQL también permite combinar métodos de particionado, lo cual se denomina particionado compuesto. Por ejemplo, se puede particionar por lista y, dentro de cada grupo, nuevamente por rango.

```
CREATE TABLE ventas_compuestas (
```

```
    id SERIAL,
```



```
region TEXT,  
fecha DATE  
) PARTITION BY LIST (region);  
CREATE TABLE ventas_norte PARTITION OF ventas_compuestas  
FOR VALUES IN ('Norte') PARTITION BY RANGE (fecha);  
CREATE TABLE ventas_norte_2020 PARTITION OF ventas_norte  
FOR VALUES FROM ('2020-01-01') TO ('2021-01-01');
```

Uno de los beneficios más evidentes de utilizar particiones es el aumento del rendimiento de las consultas. Cuando se aplican particiones, el sistema accede solo a las particiones necesarias para responder la consulta. Esto puede reducir el número de tuplas analizadas en varios órdenes de magnitud, mejorando significativamente el rendimiento general.

Sección 5: Espacios privados.

Un espacio privado en una base de datos hace referencia a una interfaz restringida o personalizada que permite a los administradores y redactores gestionar el contenido y los datos de manera más organizada y segura. En muchos sistemas, estos espacios privados se presentan en forma de paneles de control.

Los paneles de control son generalmente formularios o menús interactivos que aparecen al abrir la base de datos y actúan como un punto de entrada principal para los usuarios. Desde ahí, los usuarios pueden realizar tareas específicas como:

- Introducir nuevos datos
- Buscar registros existentes
- Generar informes
- Ejecutar procesos administrativos

Propósitos de los espacios privados

- Organización y usabilidad: Facilitan la navegación por la base de datos al ocultar elementos técnicos innecesarios para los usuarios comunes.
- Seguridad: Evitan que usuarios sin experiencia accedan directamente a las tablas, lo cual podría poner en riesgo la integridad de los datos.
- Productividad: Guían al usuario en las tareas más frecuentes, reduciendo errores y acelerando los flujos de trabajo.

Cuando se recibe una base de datos nueva, es útil comenzar explorando el panel de control. Esto puede dar pistas sobre la estructura general de la base y sobre las intenciones del diseñador con respecto al uso cotidiano del sistema.

Además, desde los vínculos del panel de control se pueden identificar rápidamente qué objetos (como formularios, informes o macros) se utilizan con mayor frecuencia.

